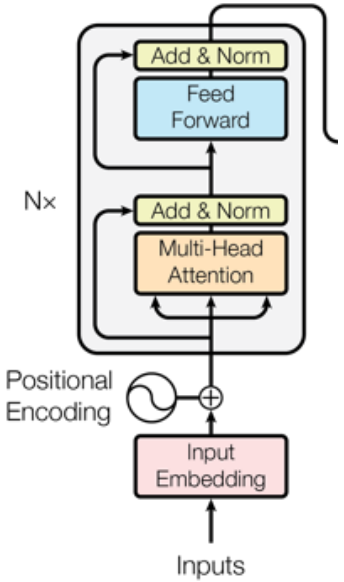
MASTER OF SCIENCE
IN ENGINEERING

Image Classification and Network Architectures II

Vision Transformer, CoAtNet, Data Augmentation and Fine Tuning

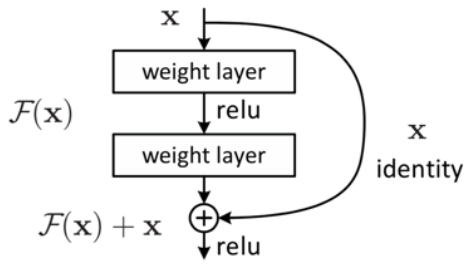


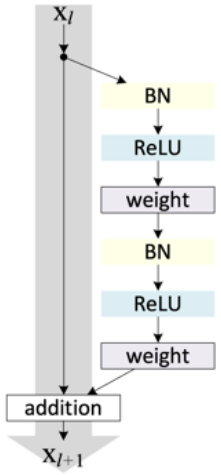
Computer Science

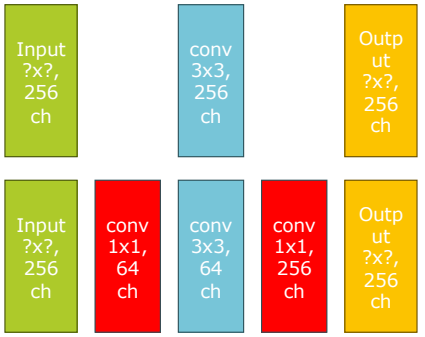
Prof. Dr. Thomas Koller

Repetition

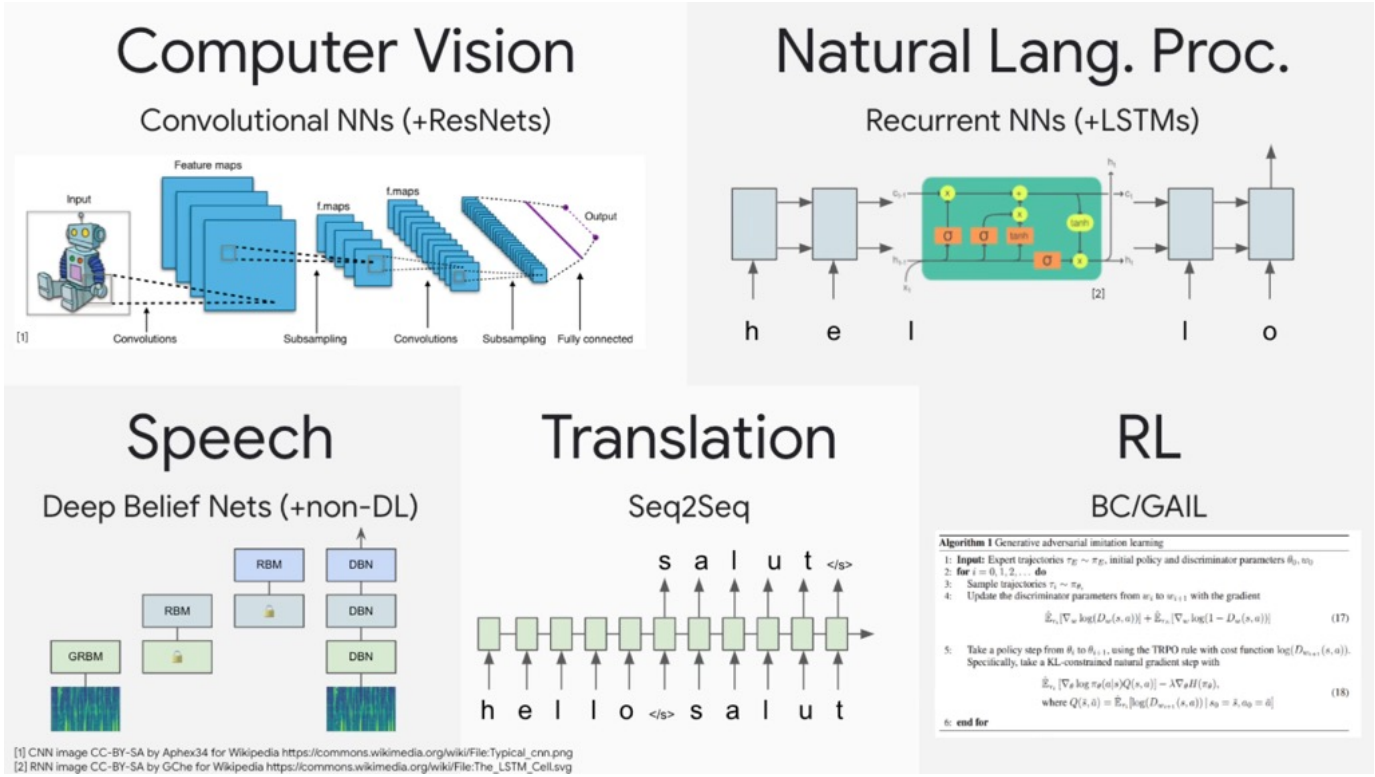
- CNNs for Image Classification requires deep neural networks
- Some mechanisms have been developed to facilitate training deep networks:
- **Skip Connections** between Layers
- **Batch Normalization**
- **Bottleneck Layers**





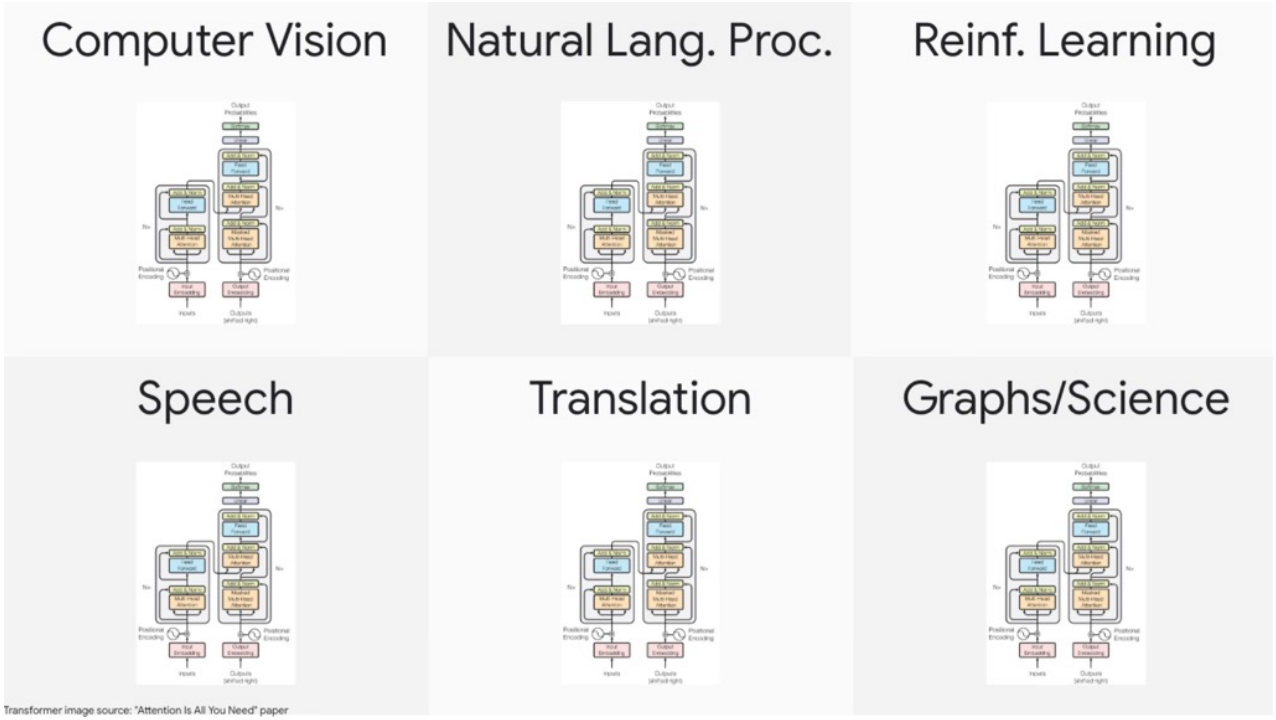


Classic Deep Learning Landscape: one architecture per 'community'



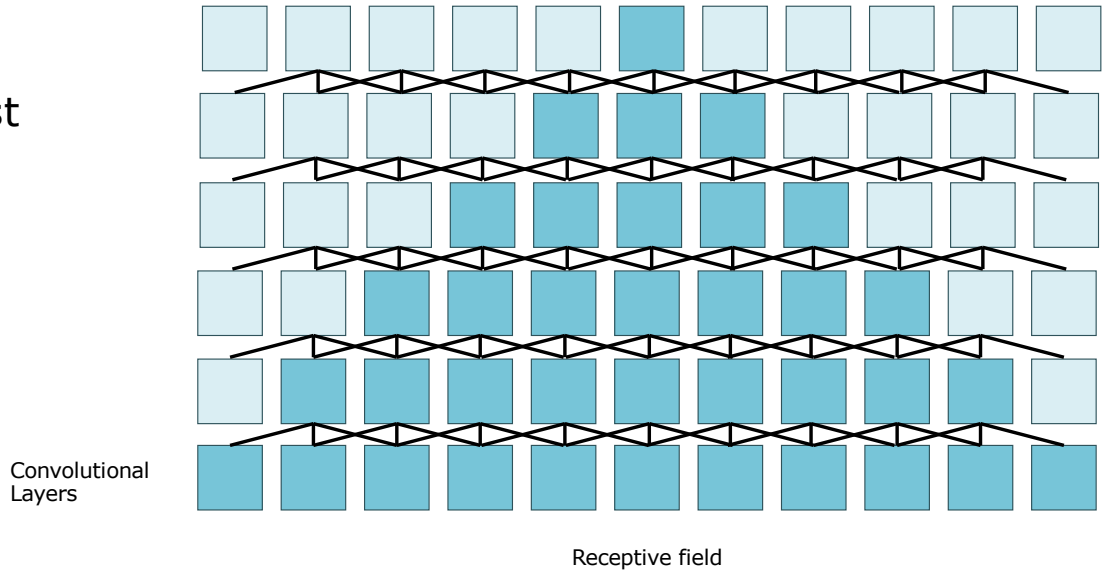
Borrowed from Lukas Beyer - Google
https://docs.google.com/presentation/d/1ZXFIhYczos679r70Yu8vV9uO6B1J0ztzeDxbnBxD1S0/edit#slide=id.g31364026ad_3_2

Transformer Takeover: One ‘community’ at a time



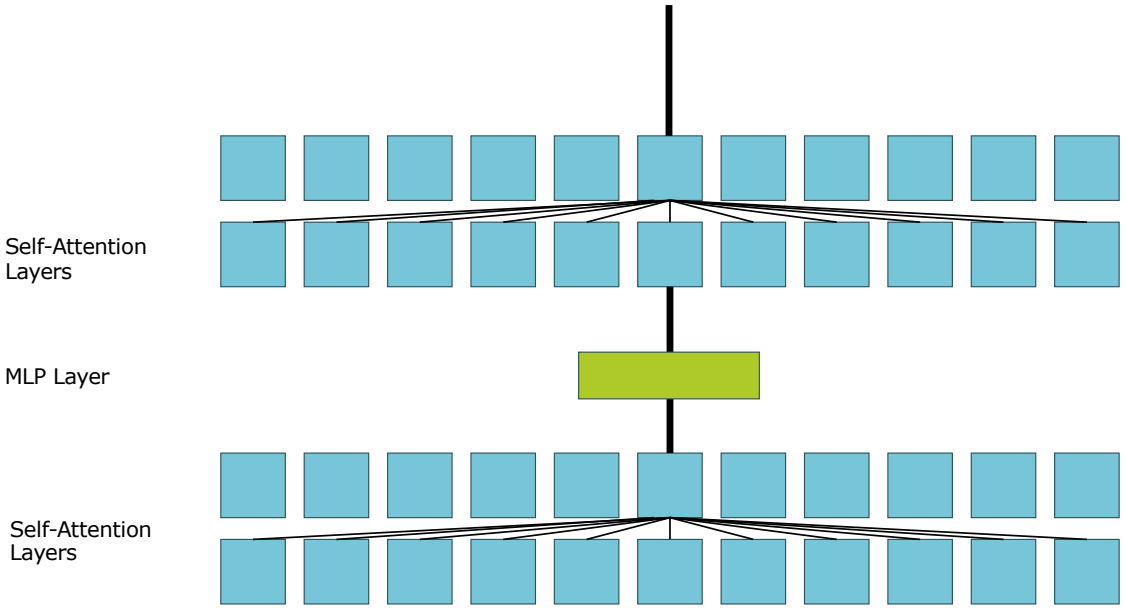
CNNS

- In CNNs, information from a location travels only slowly to other locations
- This can be sped up by decreasing the image size
- But information from the edges of the image, only reach the center by the last layers of the network
- Furthermore, the information is transformed with each layer, mixing spatial locations and channels



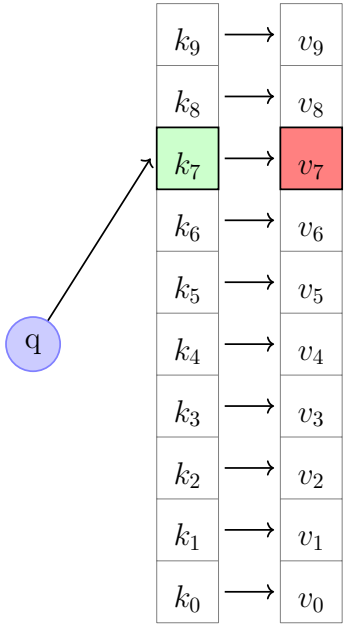
Self Attention and Transformer Architecture

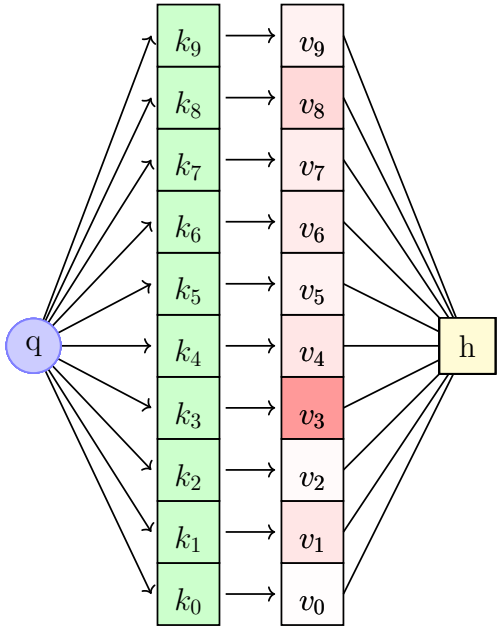
- Self-Attention layers allow to quickly access information from all of the **spatial** locations
- Then an MLP transforms the **channel information**



Attention Mechanism

Attention can be compared to the lookup in a hashtable where a value v is retrieved from a storage indexed by keys k with a query q





(Vectorized) Self-Attention Calculation

q, k and v are calculated from the same input

- With *features* stacked in X, calculate queries, keys and values:

$$Q = XW^Q \quad K = XW^K \quad V = XW^V$$

- Calculate attention scores

$$E = QK^T$$

- Calculate softmax

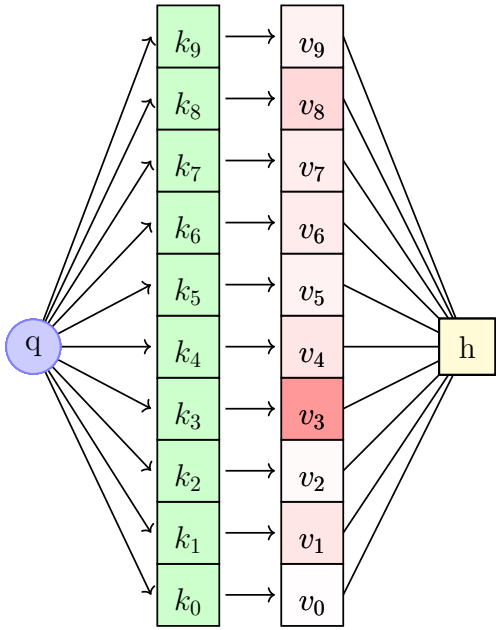
$$A = \text{softmax}(E)$$

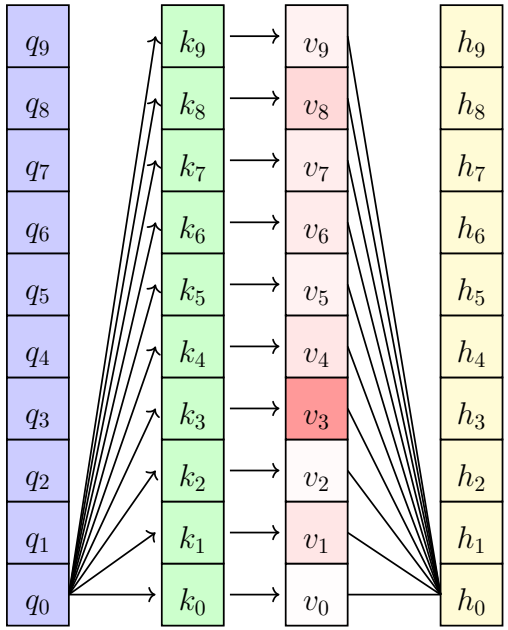
- Calculate weighted sum of the values and scale

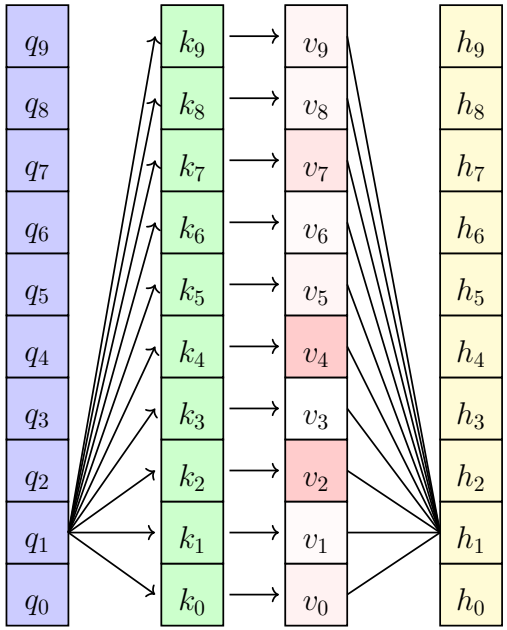
$$\text{Output} = \text{softmax}\left(QK^T/\sqrt{d_k}\right)V$$

Self-Attention

- Self-Attention calculates the attention of every query







Scaled Dot Product Attention

- After layer normalization, the mean and variance of the elements are 0 and 1
- The dot product still tends to take on extreme values as its variance scales with the dimensionality d_k
- (Variance of the sum = Sum of variance = d_k)
- To make the variance equal to 1, use:

$$\text{Output} = \text{softmax}\left(QK^T/\sqrt{d_k}\right)V$$

Attention in Keras

```
keras.layers.Attention( use_scale=False, score_mode="dot", dropout=0.0, seed=None,
**kwargs )
```

Dot-product attention layer, a.k.a. Luong-style attention.
Inputs are a list with 2 or 3 elements: 1. A query tensor of shape (`batch_size, Tq, dim`). 2. A value tensor of shape (`batch_size, Tv, dim`). 3. A optional key tensor of shape (batch_size, Tv, dim). If none supplied, value will be used as a key.
The calculation follows the steps:
1. Calculate attention scores using query and key with shape(`batch_size, Tq, Tv`).
2. Use scores to calculate a softmax distribution with shape (`batch_size, Tq, Tv`).
3. Use the softmax distribution to create a linear combination of value with shape (`batch_size, Tq, dim`).

Multiheaded Attention in Keras

```
keras.layers.MultiHeadAttention(
        num_heads, key_dim, value_dim=None,
         dropout=0.0, use_bias=True, output_shape=None, attention_axes=None,... )
```

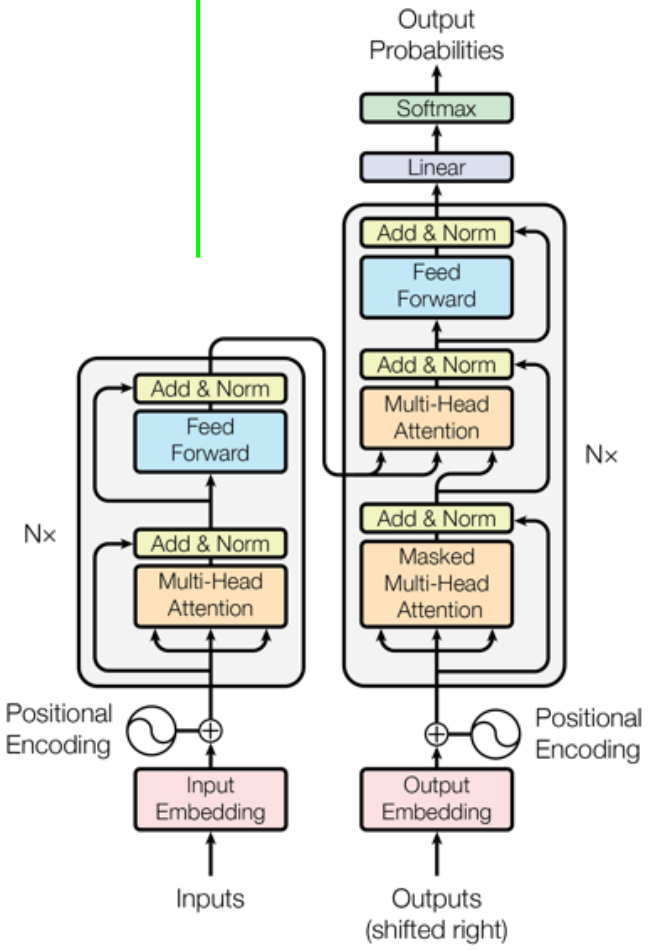
This is an implementation of multi-headed attention as described in the paper "Attention is all you Need" Vaswani et al., 2017. If `query, key, value` are the same, then this is self-attention. Each timestep in query attends to the corresponding sequence in `key,` and returns a fixed-width vector.

This layer first projects query, key and value. These are (effectively) a list of tensors of length `num_attention_heads`, where the corresponding shapes are (`batch_size, <query dimensions>, key_dim`), (`batch_size, <key/value dimensions>, key_dim`), (`batch_size, <key/value dimensions>, value_dim`).
Then, the query and key tensors are dot-producted and scaled. These are softmaxed to obtain attention probabilities. The value tensors are then interpolated by these probabilities, then concatenated back to a single tensor.

Finally, the result tensor with the last dimension as `value_dim` can take a linear projection and return.

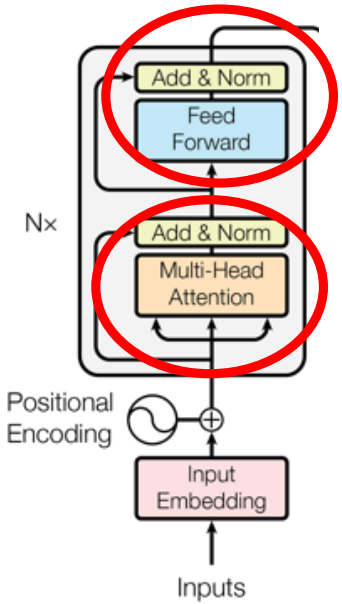
for computer vision: encoder only

Transformers



Transformer Encoder Architecture

- **Self-Attention** is the core building block of the Transformer model.
- **Multi-headed attention** is a technique to apply self-attention multiple times using projected values for Q, K and V
- Attention provides the spatial processing
- A Feed Forward layers then the channel processing



Feed Forward layer for channel processing

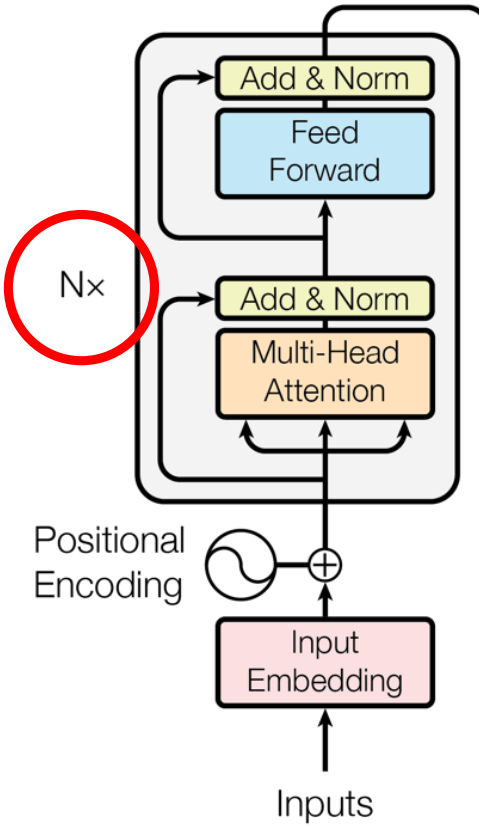
This is multi-headed **Self-Attention**

Need to go deeper…

- We would like to make these operations deeper by repeatedly stacking attention and feedforward layers.
- How to make it work for training?

Three tricks:

- Residual connections
- Layer Normalization
- Scaled Dot Product Attention

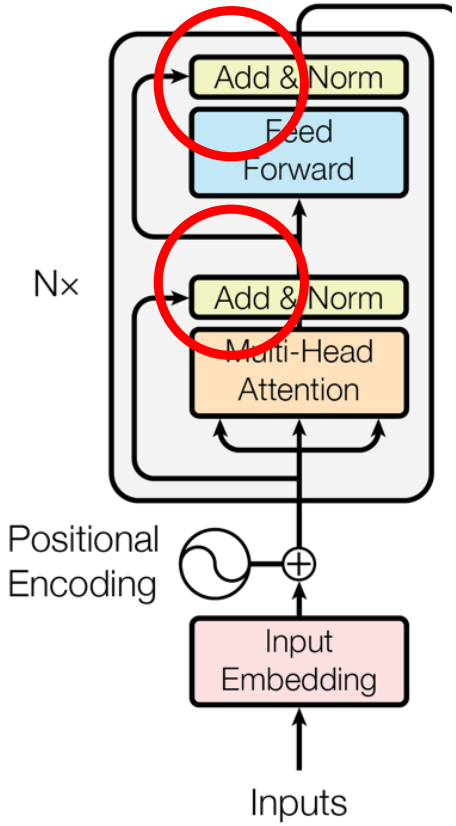


Residual connections

- Residual connections are a simple but powerful technique from ResNet (see last lecture)

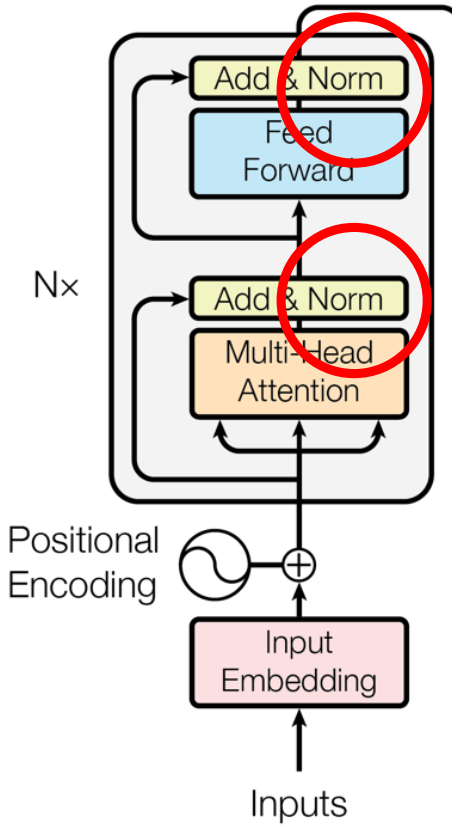
$$x_\ell = F\left(x_{\ell-1}\right) + x_{\ell-1}$$

- This prevents the network from "forgetting" or distorting important information as it is processed by many layers.



Layer Normalization

- Training can be difficult because the input of a layer keeps shifting, it is not normalized
- Reduce variation by normalizing to zero mean and standard deviation of one within each layer.
- See batch normalization (last lecture)



Position Encoding

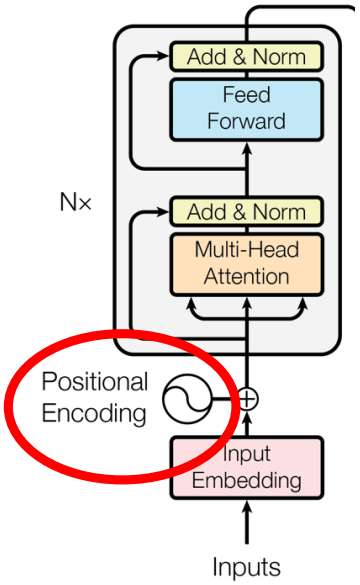
Idea: We need to add an encoding of the position of each element i of the sequence

Consider representing each sequence index as a **position vector**

$$p_i \in \mathbb{R}^d, \text{ for } i \in \{1, 2, \ldots, T\}$$

Add the position vectors to the old values, keys and queries

$$v_i = \tilde{v}_i + p_i$$
$$q_i = \tilde{q}_i + p_i$$
$$k_i = \widetilde{k}_i + p_i$$



Transformer Encoder in Keras (KerasHub)

```
keras_hub.layers.TransformerEncoder(
        intermediate_dim, num_heads,
        dropout=0, activation="relu", ...)
```

This class follows the architecture of the transformer encoder layer in the paper Attention is All You Need. Users can instantiate multiple instances of this class to stack up an encoder.

This layer will compute an attention mask, prioritizing explicitly provided masks (a `padding_mask` or a custom `attention_mask`) over an implicit Keras padding mask (for example, by passing `mask_zero=True` to a `keras.layers.Embedding` layer). If both a `padding_mask` and a `attention_mask` are provided, they will be combined to determine the final mask. See the Masking and Padding guide for more details.

Example

```
# Create a single transformer encoder layer.
encoder = keras_hub.layers.TransformerEncoder(
        intermediate_dim=64, num_heads=8)

# Create a simple model containing the encoder.
input = keras.Input(shape=(10, 64))
output = encoder(input) model = keras.Model(inputs=input, outputs=output)

# Call encoder on the inputs.
input_data = np.random.uniform(size=(2, 10, 64))
output = model(input_data)
```

Vision Transformer

Question:

- Can a standard Transformer Architecture be applied to image classification tasks?

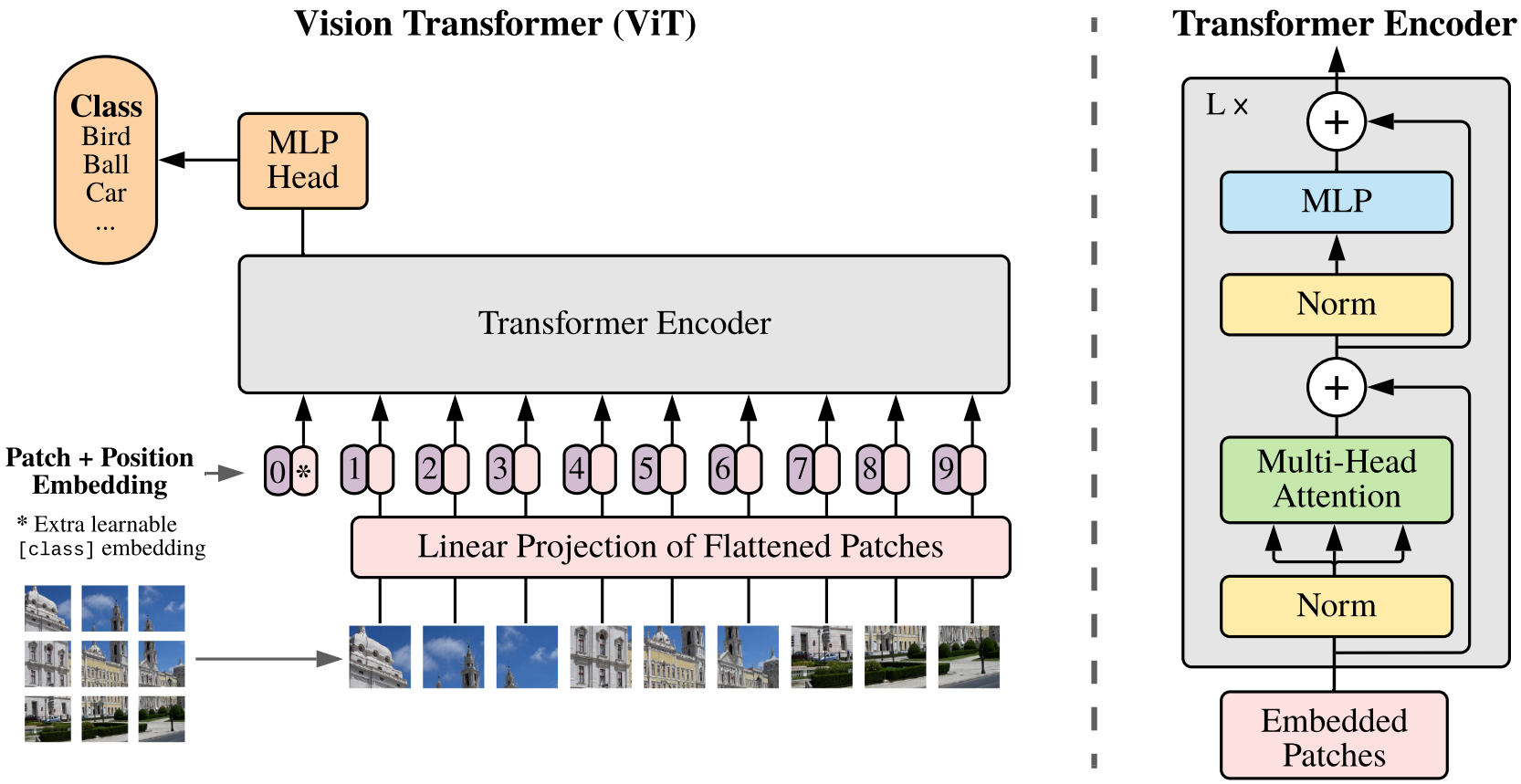
Idea:

- Split image into patches
- Calculate linear embeddings from the patches
- Use sequence of embeddings as input to a Transformer
- Train for supervised image classification

Details

- Extract square Image Patches
- Flatten Patches and project them to the embedding dimension D
- Prepend a (learnable) embedding, it’s output will serve as the output of the Transformer Encoder and input to the classification head
- Classification is implemented as a MLP with one hidden layer (pre-training) or a single linear layer (fine-tuning)
- Position embeddings (1D) are added to the input embeddings

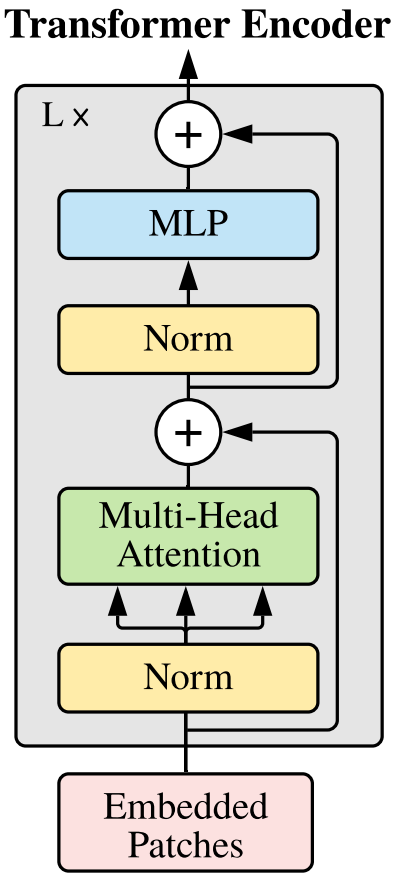
Vision Transformer (ViT)



Variants

Different models with different number of layers in the Encoder and different encoding dimensions (hidden size D)

Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

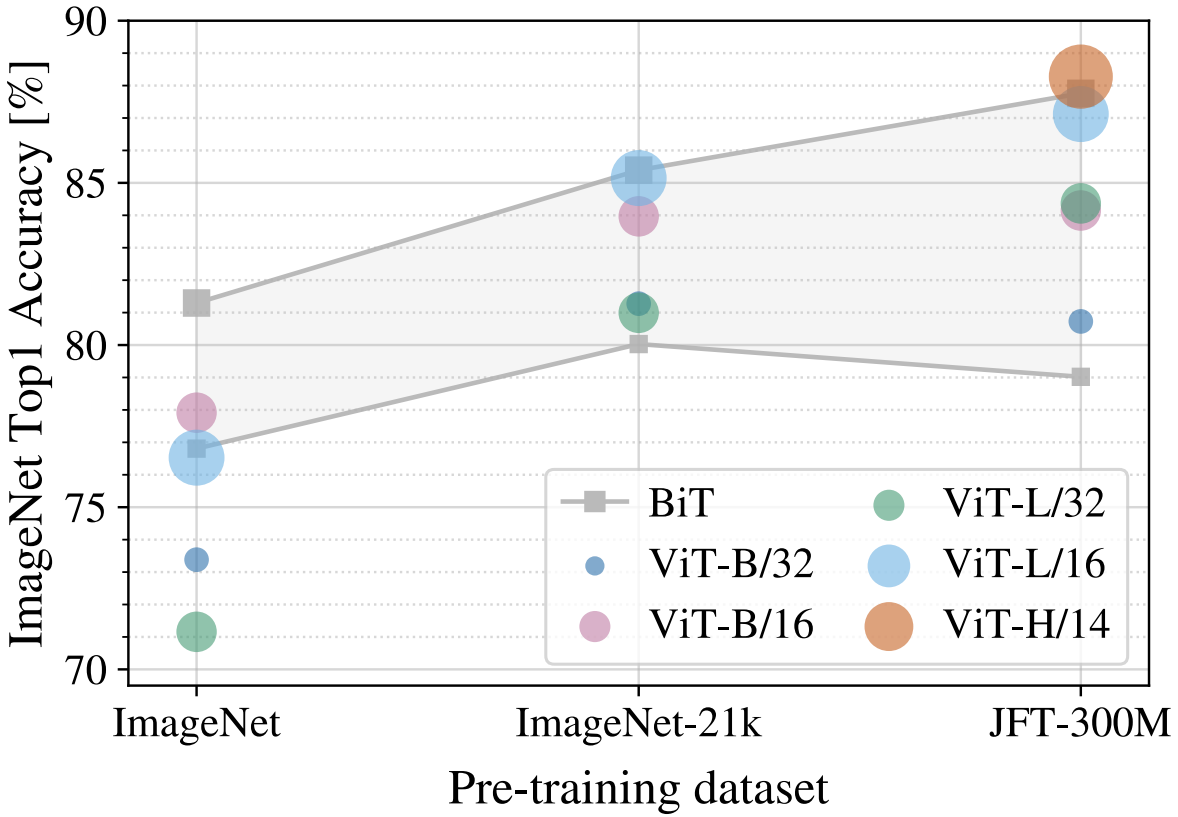


Vision Transformer

- Modest accuracies when applied to mid-sized data sets (ImageNet)
- Excellent results when pretrained on larger datasets (14M-300M) images and fine-tuned (JFT=3B

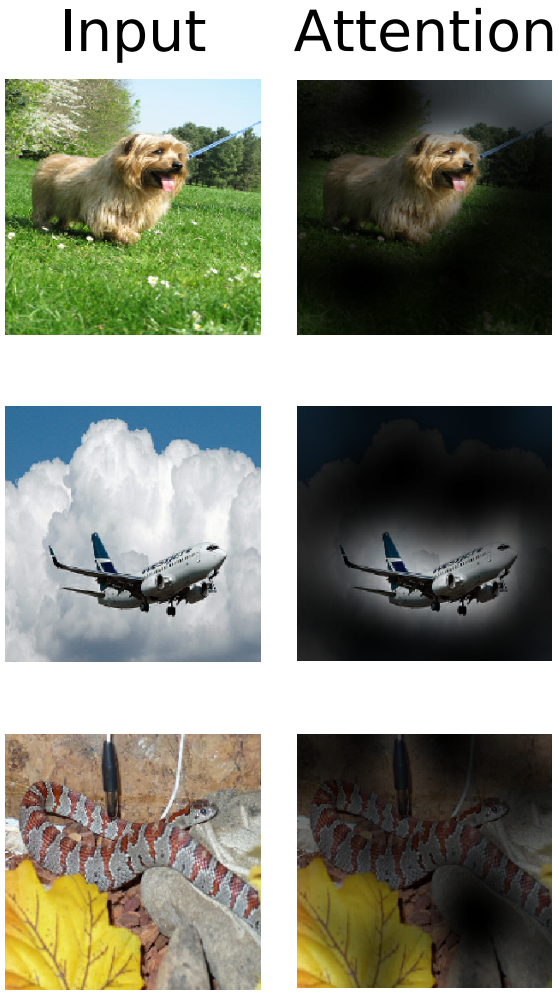
	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	**88.55** ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	**90.72** ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	**99.50** ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	–
CIFAR-100	**94.55** ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	–
Oxford-IIIT Pets	**97.56** ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	–
Oxford Flowers-102	99.68 ± 0.02	**99.74** ± 0.00	99.61 ± 0.02	99.63 ± 0.03	–
VTAB (19 tasks)	**77.63** ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	–
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

Effect on data set size



Attention Visualization

Attention weights show that the transformer concentrates on relevant parts of the image



Better still: Combination of attention and CNNs: CaAtNet

- In some of the most successful CNNs, depthwise convolution is used (see last lecture), this can be written as

$$y_i = \sum_{j \in \mathcal{L}(i)} w_{i-j} \odot x_j \qquad \text{(depthwise convolution)}$$

(where L(i) is a neighborhood of i, i.e. 3x3)

- Self-attention can be written as

$$y_i = \sum_{j \in \mathcal{G}} \underbrace{\frac{\exp\left(x_i^\top x_j\right)}{\sum_{k \in \mathcal{G}} \exp\left(x_i^\top x_k\right)}}_{A_{i,j}} x_j \qquad \text{(self-attention)},$$

Comparison

- The kernel w is **input-independent**, attention weights depend on the input dynamically. It is easier for attention to capture complicated relational interactions between spatial positions
- For (i,j) the kernel weight only depends on their relative position (translational invariance), this improves generalization. Transformers use absolute positioning.
- Attention has a global receptive field which provides more contextual information, but requires significantly more computation.

Combination

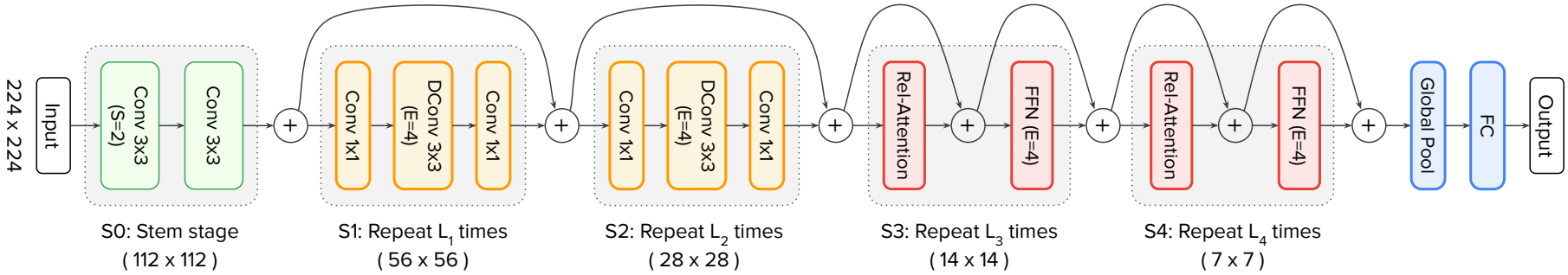
- Calculate the attention weights using the input adaptive $x_i^\top x_j$ value and the global kernel value w_{i-j}

$$y_i^{\text{pre}} = \sum_{j \in \mathcal{G}} \frac{\exp\left(x_i^\top x_j + w_{i-j}\right)}{\sum_{k \in \mathcal{G}} \exp\left(x_i^\top x_k + w_{i-k}\right)} x_j$$

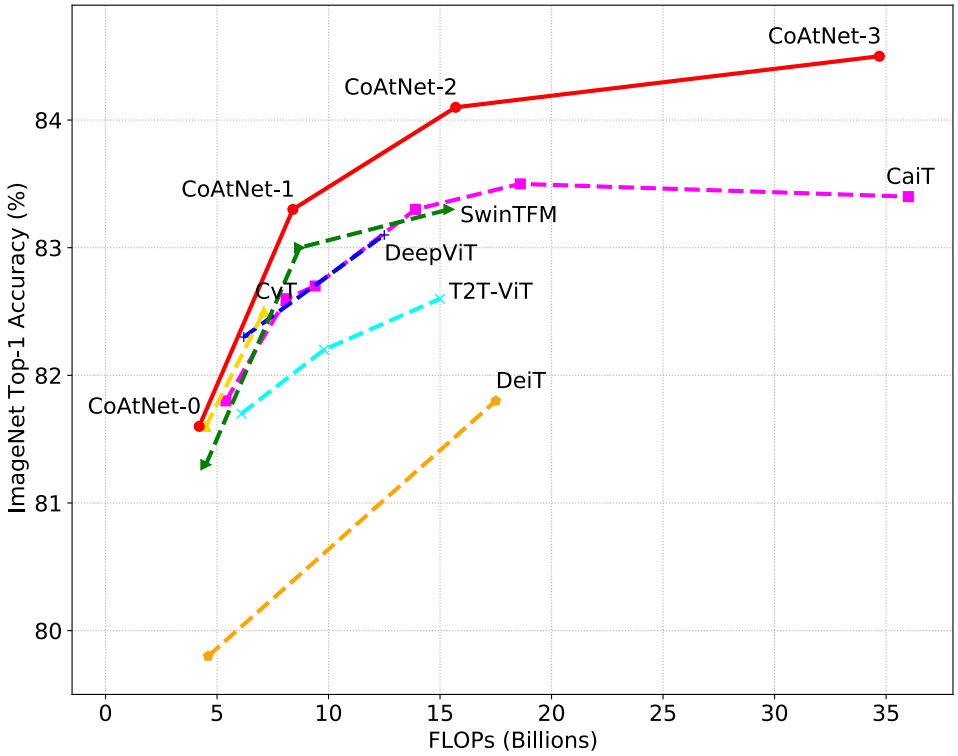
- This is used as a modified transformer block in the CoAtNet Model, it is also known as *relative self-attention*

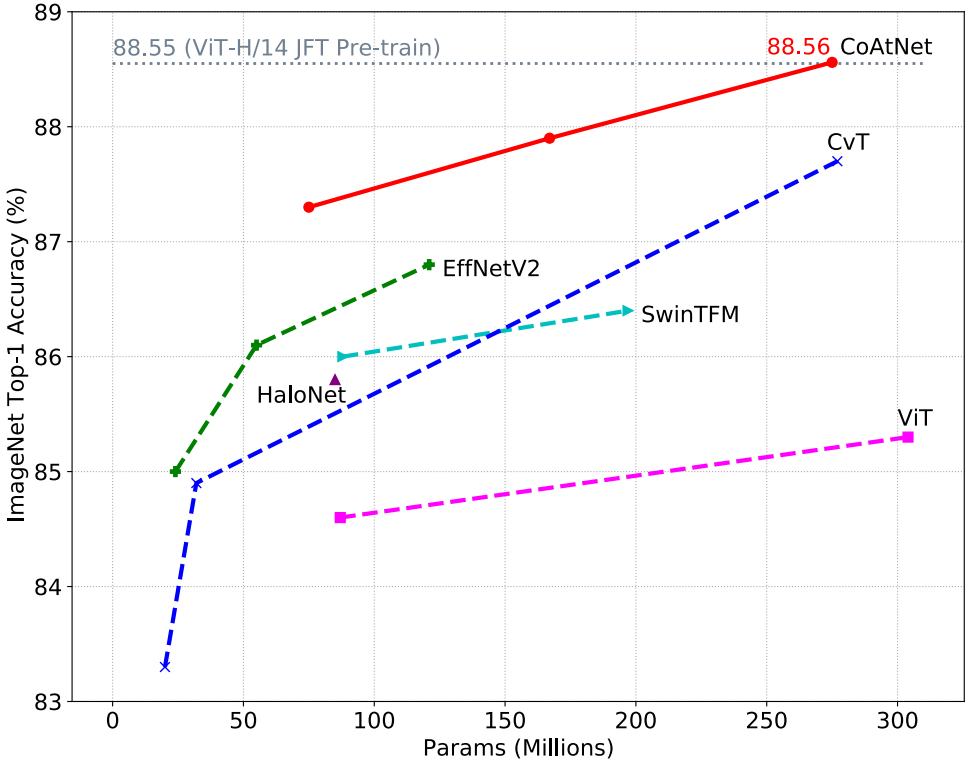
Vertical Design

- Applying the relative-attention at pixel level is computationally not possible
- A down-sampling of the image is needed (for example with strided convolutions)
- Mimic CNNs architecture: use different stages and down-sample at the beginning of each stage while increasing the number of channels
- Tests showed best results (for generalization, model capacity and transferability) with 3 convolution blocks, followed by 2 transformer blocks



Results





Comparison of accuracy vs FLOPs and Accuracy vs. Nr Parameters

Convolutional Neural Networks in Practice: Augmenting your data set

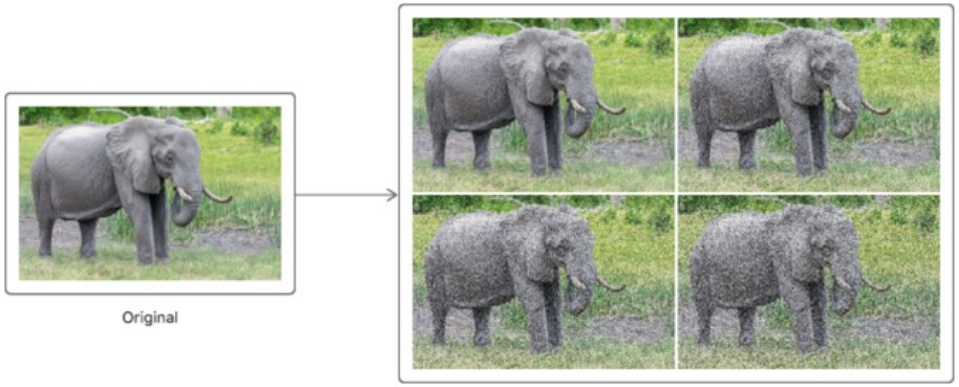
The discussed models were trained on the **ImageNet dataset**, which contains more than **14 million labeled images**!

In practice, you rarely have access to labeled datasets of this size.

What to do?

- Augmenting your Dataset

Source: https://medium.com/analytics-vidhya/data-augmentation-is-it-really-necessary-b3cb12ab3c3f



Source: https://developer.apple.com/documentation/createml/mlimageclassifier/imageaugmentationoptions/noise

Image Augmentation

- Applied only on the training set, not validation or test
- Implemented as part of the model (less space, more training time) or by generating an augmented version of the data set
- Typically with parameters giving the amount of augmentation (which is then a kind of hyper parameter to the training)

https://github.com/aleju/imgaug

Types of Augmentation

Geometric:

- Rotation
- Scaling
- Cropping
- Shear
- Flip horizontal/vertical

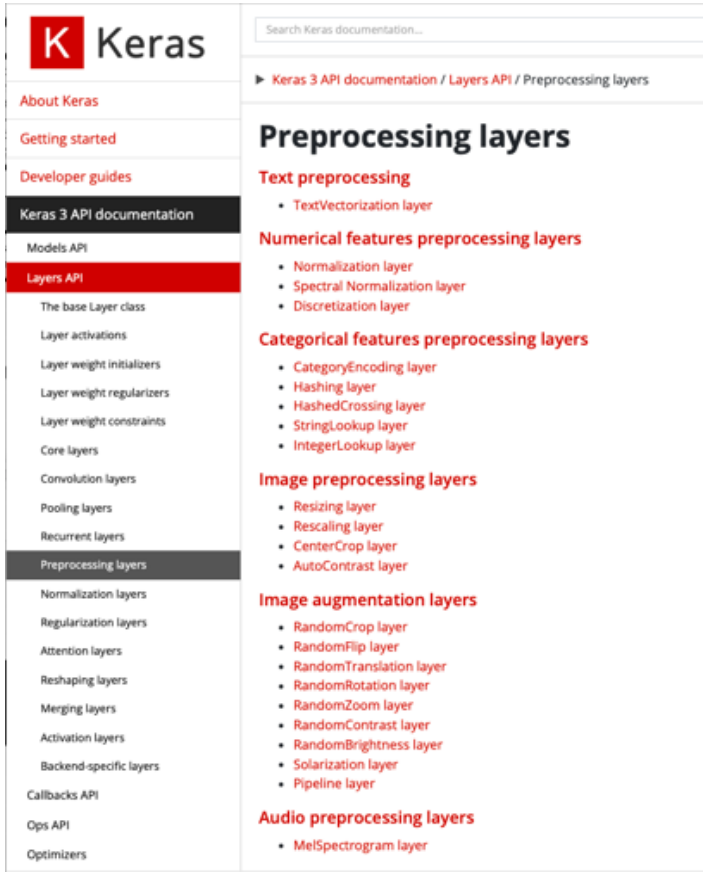
Color:

- Contrast
- Brightness
- Color Adjustments

Augmentation Layers in Keras

```python
def create_preprocessing_pipeline():
    preprocessing_pipeline = keras.Sequential([
        keras.layers.RandomContrast(0.05),
        keras.layers.RandomZoom(0.05),
        keras.layers.RandomRotation(0.05),
        keras.layers.RandomBrightness(0.05),
    ])
    return preprocessing_pipeline


preprocessing = create_preprocessing_pipeline()
model = keras.Sequential()
model.add(preprocessing)
model.add(base_model)
```



Auto Augmentation

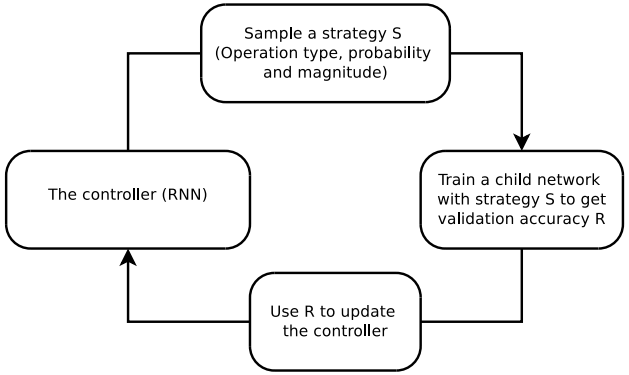
AutoAugment:
Learning Augmentation Strategies from Data

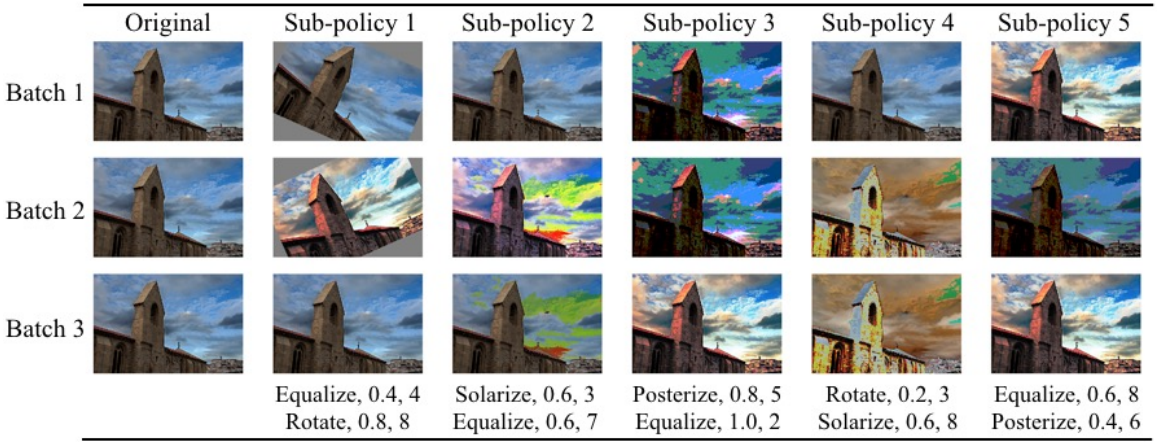
Ekin D. Cubuk *, Barret Zoph*, Dandelion Mané, Vijay Vasudevan, Quoc V. Le
Google Brain

It can be difficult to find the optimal methods and parameters for the augmentation:

AutoAugment approach:

- Automatically search for data augmentation policies





- Best policies on ImageNet favor Color Augmentation

Convolutional Neural Networks in Practice: Fine-tuning existing models

The discussed models were trained on the **ImageNet dataset**, which contains more than **14 million labeled images**!

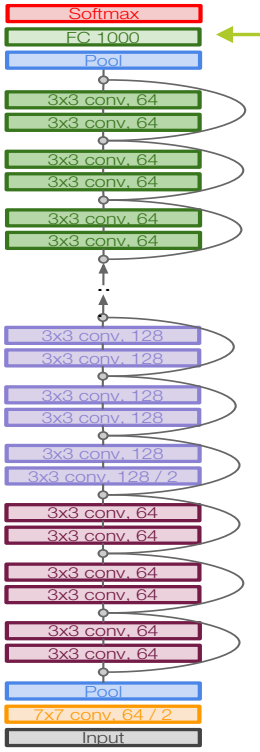
In practice, you rarely have access to labeled datasets of this size.

What to do?

- Fine-tuning an existing model

Available models

Model	Size	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth
Xception	88 MB	0.790	0.945	22,910,480	126
VGG16	528 MB	0.713	0.901	138,357,544	23
VGG19	549 MB	0.713	0.900	143,667,240	26
ResNet50	98 MB	0.749	0.921	25,636,712	-
ResNet101	171 MB	0.764	0.928	44,707,176	-
ResNet152	232 MB	0.766	0.931	60,419,944	-
ResNet50V2	98 MB	0.760	0.930	25,613,800	-
ResNet101V2	171 MB	0.772	0.938	44,675,560	-
ResNet152V2	232 MB	0.780	0.942	60,380,648	-
InceptionV3	92 MB	0.779	0.937	23,851,784	159
InceptionResNetV2	215 MB	0.803	0.953	55,873,736	572
MobileNet	16 MB	0.704	0.895	4,253,864	88
MobileNetV2	14 MB	0.713	0.901	3,538,984	88
DenseNet121	33 MB	0.750	0.923	8,062,504	121
DenseNet169	57 MB	0.762	0.932	14,307,880	169
DenseNet201	80 MB	0.773	0.936	20,242,984	201
NASNetMobile	23 MB	0.744	0.919	5,326,716	-
NASNetLarge	343 MB	0.825	0.960	88,949,818	-
EfficientNetB0	29 MB	-	-	5,330,571	-
EfficientNetB1	31 MB	-	-	7,856,239	-
EfficientNetB2	36 MB	-	-	9,177,569	-
EfficientNetB3	48 MB	-	-	12,320,535	-
EfficientNetB4	75 MB	-	-	19,466,823	-
EfficientNetB5	118 MB	-	-	30,562,527	-
EfficientNetB6	166 MB	-	-	43,265,143	-
EfficientNetB7	256 MB	-	-	66,658,687	-



Retrain only this layer with your data

- Learning few parameters requires few training samples
- Assumption: the ”stem” of the network learned to extract generally useful features

Reuse as-is

Transfer Learning

- A trained model for image classification on a large data set is expected to have learned feature extraction that is relevant to any image classification tasks
- A typical architecture consists of convolutional layers, followed by a classification head with one or more fully connected layers
- If we just use the convolutional layers, they will do the **feature extraction** and we can attach a new head for the classification (most likely for a different number of classes)
- The convolutional layers should be **frozen**, so that their weights are not updated
- They can be unfrozen and updated at the end using a very small learning rate, this is called **fine-tuning**

Caveats

- Base networks might have been trained on different image sizes
- They often work for a range of sizes if the classification head is excluded, but not for arbitrary sizes, for example InceptionV3 works for sizes > 75, others for sizes as small as 32
- If the input image are much larger (or much smaller) than they were trained for (for example 4k), they might not perform as well, and the output features will be too large

Transfer Learning in Keras

- Layers (and models) have an attribute **`trainable`**, that can be set to **`False`**
- This will be applied recursively to all sublayers
- The attribute has to be changed before calling **`compile`** on the model
- https://keras.io/guides/transfer_learning/

- Instead of incorporating the base network in your model, you can also run it once and extract and save the features
- Then train a new, small model with those features as input
- (This is more efficient, but less flexible and requires more space)

Exercise

- Add Data Augmentation to your model for CIFAR-100

- Adapt an existing model to CIFAR-100
- How does it perform without pretrained weights?
- Use pretrained weights, freeze layers and retrain only classification head

```
keras.applications.ResNet50V2(
    include_top=True,
    weights="imagenet",
    input_tensor=None,
    input_shape=None,
    pooling=None,
    classes=1000,
    classifier_activation="softmax”,
    name="resnet50v2", )
```